

Question 1

Score: 2/2

In the OLTP Database lab, specifically Exercise 3, Task 5, what query should you use to find out how many entries exist in the `sales_data` table?

```
SELECT COUNT(*) FROM sales_data;
```

Question 2

Score: 0/2

Enter the contents of the file **datadump.sh** created as part of Exercise 4, Task 8 of the **OLTP** lab, which extracts all the rows from the **sales_data** table into the **sales_data.sql** file.

```
#!/bin/bash
```

```
# Database credentials
```

```
DB_HOST="localhost"
```

```
DB_USER="myuser"
```

```
DB_PASSWORD="mypassword"
```

```
DB_NAME="sales"
```

```
TABLE_NAME="sales_data"
```

```
OUTPUT_FILE="sales_data.sql"
```

```
# Export the table to a .sql file using mysqldump
```

```
mysqldump -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASSWORD" "$DB_NAME"  
"$TABLE_NAME" > "$OUTPUT_FILE"
```

```
# Check if the command was successful
```

```
if [ $? -eq 0 ]; then
```

```
    echo "Data successfully exported to $OUTPUT_FILE"
```

```
else
```

```
    echo "Error exporting data"
```

```
fi
```

Question 3

Score: 0/1

Refer to the screenshot **softcartRelationships.jpg** and the associated code saved as text in Exercise 1, Task 6 of the **Data Warehouse** Lab. How many direct connections does the **softcartFactSales** table have with other tables?

5

Question 4

Score: 0/3

Refer to the screenshot **groupingsets.jpg** created in Task 5 of the **Data Warehouse Reporting** lab. Enter the SQL command used to create a **GROUPING SETS** query using the **country**, **category**, and **total_sales** columns.

```
SELECT
    dc.country,
    dt.category,
    SUM(f.amount) AS total_sales
FROM
    FACTSALES AS f
JOIN
    DIMCOUNTRY AS dc ON f.countryid = dc.countryid
JOIN
    DIMCATEGORY AS dt ON f.categoryid = dt.categoryid
GROUP BY
    GROUPING SETS (
        (dc.country),
        (dt.category)
    );
```

Question 5

Score: 0/3

Refer to the screenshot **rollup.jpg** and the associated code saved as text in Task 6 of the **Data Warehouse Reporting** lab. Enter the SQL command used to create a **ROLLUP** query using the **year**, **country**, and **total_sales** columns.

```
SELECT dd.year, dc.country, SUM(f.amount) AS totalsales
FROM FACTSALES AS f
JOIN DIMCOUNTRY AS dc ON f.countryid = dc.countryid
JOIN DIMDATE AS dd ON f.dateid = dd.dateid
GROUP BY ROLLUP(dd.year, dc.country)
ORDER BY dd.year, dc.country
```

Question 6

Score: 0/3

Refer to the screenshot **cube.jpg** and the associated code saved as text in Task 7 of the **Data Warehouse Reporting** lab. Enter the SQL command used to create a **CUBE** query using the **year**, **country**, and **average_sales** columns.

```
SELECT dd.year, dc.country, AVG(f.amount) AS average_sales
FROM FACTSALES AS f
JOIN DIMCOUNTRY AS dc ON f.countryid = dc.countryid
JOIN DIMDATE AS dd ON f.dateid = dd.dateid
GROUP BY CUBE(dd.year, dc.country)
ORDER BY dd.year, dc.country
```

Question 7

Score: 0/3

Refer to the screenshot **mqt.jpg** and the associated code saved as text in Task 8 of the **Data Warehouse Reporting** lab. Enter the SQL command used to create an MQT named **total_sales_per_country** with the **country** and **total_sales** columns.

```
CREATE TABLE total_sales_per_country(country, total_sales) AS (
    SELECT DIMCOUNTRY.country, SUM(FACTSALES.amount) AS total_sales
    FROM FACTSALES
    LEFT JOIN DIMCOUNTRY ON FACTSALES.countryid = DIMCOUNTRY.countryid
```

GROUP BY DIMCOUNTRY.country

)

Question 8

Score: 0/5

What is the SQL query used in Exercise 1, Task 1 of the **ETL** lab for implementing the **get_row_id()** function that returns the last row ID of the **sales_data** table?

def get_last_rowid():

```
SQL = "SELECT MAX(ROWID) FROM sales_data"
```

```
stmt = ibm_db.exec_immediate(conn, SQL)
```

```
res = ibm_db.fetch_both(stmt)
```

```
print(res)
```

```
return int(res[0])
```

Question 9

Score: 0/5

What is the code snippet used in Exercise 1, Task 2 of the **ETL** lab, while implementing the **get_latest_records()** function that returns the latest records of the **sales_data** table, that is, with row id greater than a passed argument?

def get_latest_records(rowid):

```
SQL = "SELECT * FROM sales_data WHERE rowid > %s"
```

```
cursor.execute(SQL, [rowid])
```

```
new_recs = cursor.fetchall()
```

```
# for row in new_recs: print(row)
```

```
return new_recs
```

Question 10

Score: 0/5

What is the code snippet used in Exercise 1, Task 3 of the **ETL** lab for implementing the **insert_records()** function that inserts all the given records into the **sales_data** table?

```
def insert_records(records):
```

```
    SQL = "INSERT INTO sales_data(rowid,product_id,customer_id,quantity) VALUES(?,?,?,?);"
    stmt = ibm_db.prepare(conn, SQL)
```

```
    for record in records:
```

```
        ibm_db.execute(stmt, record)
```

Question 11

Score: 0/3

Enter the Python code block that defines the configuration to be used for the DAG in Exercise 2, Task 1 of the **Data Pipelines Using Apache Airflow** lab.

```
from airflow.utils.dates import days_ago
```

```
default_args = {
```

```
    'owner': 'Dennis',
```

```
    'start_date': days_ago(0),
```

```
    'email': ['dennis@mail.com']
```

```
}
```

Question 12

Score: 0/3

Enter the Python code block that defines the DAG in Exercise 2, Task 2 of the **Data Pipelines Using Apache Airflow** lab.

```
from airflow import DAG
```

```
from datetime import timedelta
```

```
from airflow.utils.dates import days_ago
```

```
default_args = {
```

```
    'owner': 'Dennis',
```

```
    'start_date': days_ago(0),
```

```
'email': ['dennis@mail.com']  
}
```

```
dag = DAG(  
    dag_id='process_web_log',  
    default_args=default_args,  
    description='A DAG to process web logs daily',  
    schedule_interval=timedelta(days=1)  
)
```

Question 13

Score: 0/3

Enter the Python code block that extracts the IP address field from the web server log file and saves it to a file named **extracted_data.txt** in Exercise 2, Task 3 of the **Data Pipelines Using Apache Airflow** lab.

```
from airflow.operators.bash import BashOperator
```

```
extract_data = BashOperator(  
    task_id='extract_data',  
    bash_command="cut -d ' ' -f1 accesslog.txt > extracted_data.txt",  
    dag=dag,  
)
```

Question 14

Score: 0/3

Enter the Python code block that filters out all occurrences of the IP address **198.46.149.143** from the **extracted_data.txt** file and saves the output to a file named **transformed_data.txt** in Exercise 2, Task 4 of the **Data Pipelines Using Apache Airflow** lab.

```
from airflow.operators.bash import BashOperator
```

```
transform_data = BashOperator(  
    task_id='transform_data',
```

```
bash_command="grep -v '198.46.149.143' extracted_data.txt > transformed_data.txt",
dag=dag,
)
```

Question 15

Score: 0/3

Enter the Python code block that archives the file **transformed_data.txt** into a tar file named **weblog.tar** in Exercise 2, Task 5 of the **Data Pipelines Using Apache Airflow** lab.

```
from airflow.operators.python import PythonOperator
```

```
import tarfile
```

```
def archive_file():
```

```
    with tarfile.open('weblog.tar', mode='w') as archive:
```

```
        archive.add('transformed_data.txt')
```

```
load_data = PythonOperator(
```

```
    task_id='load_data',
```

```
    python_callable=archive_file,
```

```
    dag=dag,
```

```
)
```

Question 16

Score: 0/2

Enter the MySQL terminal command used to create a backup of the **FactSales** table to the file **sales_backup.sql**, specifying port 3306 and requiring manual password entry, as part of Task 1 in the **Backup and Restore** lab.

```
mysqldump -u root -p -P 3306 your_database_name FactSales > sales_backup.sql
```

Question 17

Score: 0/2

Enter the MySQL terminal command used to restore the **FactSales** table from the file **sales_backup.sql**, specifying port **3306** and requiring manual password entry, as part of Task 3 in the **Backup and Restore** lab.

```
mysql -u root -p -P 3306 -D sales < sales_backup.sql
```

Question 18

Score: 0/7

Enter the contents of the bash script **backup_automation.sh** as a part of Task 2 in the **Backup and Restore Automation** lab. This script should:

1. Create the database backup as a zip file. The nomenclature of the saved backup should be **backup_sales_<timestamp>.gz**.
2. Check if the backup directory exists; if not, create it.
3. Set the backup retention period to 10 days and delete the old backups.

```
#!/bin/bash
```

```
# === Configuration ===
```

```
DB_NAME="sales"
```

```
BACKUP_DIR="$HOME/db_backups"
```

```
TIMESTAMP=$(date +%Y%m%d_%H%M%S')
```

```
BACKUP_FILE="backup_${DB_NAME}_${TIMESTAMP}.gz"
```

```
RETENTION_DAYS=10
```

```
# === Ensure backup directory exists ===
```

```
if [ ! -d "$BACKUP_DIR" ]; then
```

```
    echo "Backup directory does not exist. Creating: $BACKUP_DIR"
```

```
    mkdir -p "$BACKUP_DIR"
```

```
fi
```

```
# === Check if database exists ===
```

```
DB_EXISTS=$(mysql -u root -e "SHOW DATABASES LIKE '${DB_NAME}';" | grep  
"${DB_NAME}")
```

```
if [ "$DB_EXISTS" != "$DB_NAME" ]; then
```

```
    echo "✗ Database '${DB_NAME}' does not exist. Backup aborted."
```

```
    exit 1
```



```

fi

# === Perform the backup ===

echo "Creating backup for database '${DB_NAME}'..."

mysqldump -u root "$DB_NAME" | gzip > "$BACKUP_DIR/$BACKUP_FILE"

if [ $? -eq 0 ]; then

    echo "✓ Backup successful: $BACKUP_DIR/$BACKUP_FILE"

else

    echo "✗ Backup failed!"

    exit 1

fi

# === Clean up old backups ===

echo "Cleaning up backups older than $RETENTION_DAYS days..."

find "$BACKUP_DIR" -name "backup_${DB_NAME}_*.gz" -type f -mtime
+$RETENTION_DAYS -exec rm -f {} \;

echo "✓ Done."

```

Question 19

Score: 0/3

Enter the crontab command used to schedule the **backup_automation.sh** script to run every three minutes, every day, as part of Task 3 in the **Backup and Restore Automation** lab.

```

*/3 * * * * /home/project/.theia/backup_automation.sh >>
/home/project/.theia/backup_cron.log 2>&1

```

Question 20

Score: 0/3

Enter the MySQL terminal commands that were used to unzip the latest backup file and restore the database as part of Task 4 in the **Backup and Restore Automation** lab.

```
gunzip -k "$LATEST_BACKUP"
```

```
mysql -u root sales < "${LATEST_BACKUP%.gz}"
```

Question 21

Score: 0/3

Enter the MySQL CLI command used to create the index **country_id** on the **countryid** field in the **FactSales** table, as part of Task 1 in the **Database and Query Optimization** lab.

```
CREATE INDEX country_id ON FactSales (countryid);
```

Question 22

Score: 1/1

Refer to the screenshot **final_data_types.jpg** created as a part of Task 2 in the **Database and Query Optimization** lab. What is the ideal length of the **Monthname** field, based on the contents of the **DimDate** table?

9

Question 23

Score: 0/3

Enter the command used to modify the data type of the amount field in the **FactSales** table to a variable length binary with a maximum size of 255 bits, as used in Task 2 of the **Access Management and Database Security** lab.

```
ALTER TABLE FactSales MODIFY amount VARBINARY(255);
```

Question 24

Score: 0/3

Enter the command used to encrypt the data in the amount field of the **FactSales** table using the hashed passphrase, as used in Task 2 of the **Access Management and Database Security** lab.

```
SET @key = SHA2('sales info encryption', 512);
```

```
UPDATE FactSales
```

```
SET amount = AES_ENCRYPT(CAST(amount AS CHAR), @key);
```

Question 25

Score: 1/1

What is the difference between the output of the data query executed with and without decryption, as used in the last two steps of Task 2 of the **Access Management and Database Security** lab?

With decryption, the amount column will have the actual data displayed; without decryption, the column is displayed having encoded and unintelligible values.